

Retraining the Model

Local retrain, zero permanent local data · Sep 18 2026

Why local, not Render: a single leave-one-round-out CV fold took over 6 minutes on Render's free-tier instance — confirmed live, not a guess. A full retrain there would run well over an hour. Your Mac's CPU does the same work in minutes, so retraining always happens locally.

Why one script, not the old manual sequence: the old path (edit features locally, run `prepare_phase3_dataset.py`, run `retrain_pipeline.py`) silently trained on stale data once round data started being fetched entirely through the deployed app — your Mac's copy of `features.parquet` / `qualifying_targets.parquet` never saw those rounds. A local retrain "with Round 14" on Sep 16 2026 reproduced the previous model exactly, because the local data still stopped at Round 13. `scripts/retrain_from_hf.py` closes that gap: it pulls the current data from Hugging Face first, every time, so the local retrain always sees what the app has already fetched — and it deletes its own local copies afterward, so nothing accumulates on your Mac between retrains.

THE ONE COMMAND

Run from the repo root, on your Mac (not through the app). `HF_DATASET_REPO` and `HF_TOKEN` are the same two values Render already uses — copy them from the Render dashboard: open the `f1qp_render_deploy` service, open its Environment tab, and reveal each value there.

```
HF_DATASET_REPO=<paste from Render> HF_TOKEN=<paste from Render> \  
python scripts/retrain_from_hf.py
```

WHAT IT DOES, IN ORDER

1. Pull

Downloads the current `features.parquet` and `qualifying_targets.parquet` from Hugging Face into `data/processed/` — whatever the app has already fetched, including rounds you've never seen locally. Prints the 2026 rounds now present, so you can confirm the round you expect (e.g. 14) is actually there before it retrains.

2. Rebuild the dataset

Runs the same dataset-assembly step as before (`prepare_phase3_dataset.py`, in-process) on that freshly pulled data.

3. Retrain

Runs `retrain_pipeline.py` in-process. Writes a staged candidate under `models/lstm/staged/<timestamp>/` — never touches the live, currently-promoted model. Logs the run to the local MLflow store.

4. Push

Pushes the staged candidate and the MLflow run back to Hugging Face.

5. Verify

Re-lists the Hugging Face repo's files and checks every pushed path actually landed there — doesn't just trust that the push didn't error.

6. Clean up

Only if verification passed: deletes the local features/targets/dataset files, the pulled launch JSONs, and the staged candidate directory. Your Mac ends the run with no more retrain data on disk than it started with. The live model under `models/lstm/` and the local MLflow database are left alone — they aren't inputs this run pulled, and other local tooling still expects them.

IF VERIFICATION FAILS

Nothing local gets deleted. The script exits, the staged candidate stays exactly where `retrain_pipeline.py` left it, and the script prints why the check failed. Re-running it is safe — worst case, it retrains a second time. The failure mode here is always “keep the local copy one extra cycle,” never “lose the retrain.”

AFTER IT FINISHES

The new candidate is on Hugging Face, not yet live — Render only reads from Hugging Face when its container starts. Two manual steps, both on purpose, both yours to trigger:

- **Restart the Render service.** Render dashboard → `f1qp_render_deploy` → Manual Deploy → Restart service. Takes about 10 seconds. This is a restart, not a rebuild — it does not touch your code, only reloads the container so it pulls the new staged candidate from Hugging Face.
- **Promote to production, in the app.** The app's own “Promote to production” action is the explicit checkpoint that makes the new candidate the one actually serving predictions. Nothing goes live on its own before this.

IF YOU'RE DEPLOYING A CODE CHANGE INSTEAD

Different operation, different command — noted here because it's easy to confuse the two. A code change (backend or frontend, e.g. a frontend/src edit) is not picked up by a restart, and there is no `npm` command to run yourself — `npm` only runs inside Render's own Docker build, which is why “`npm run build`” at the repo root fails with “command not found.” The one command that ships a code change is:

```
bash deploy/render/sync_and_push.sh
```

Run from the repo root, on your Mac. It assembles a fresh build context and force-pushes it to the private GitHub repo Render is connected to; Render detects the push and rebuilds on its own — that rebuild is what actually runs `npm install / npm run build` for the frontend, inside the Docker image. A full rebuild takes a few minutes (installing torch is the slow part); watch the service's Events or Logs tab on the Render dashboard.

Quick reference: retrain → restart service → Promote. Code change → `sync_and_push.sh`.